

VEILORA — Integrated Product Approach

Threshold control, private execution, and explainable digital finance

Status: Proposed integrated direction

Inputs: Veilora PRD v1; shielded-finance architecture research

Initial network: Robinhood Chain, Chain ID 4663

Working brand line: Move quietly. Stay in control.

Important: This document is a product and architecture proposal. Claims about market position, competitor behavior, audits, privacy guarantees, tokenized-stock standards, proof-provider availability, and production readiness must be independently verified before public use.

1. The integrated thesis

Veilora should combine two systems that are individually strong but much more valuable together:

1. **Control Plane:** threshold self-custody, intent planning, simulations, policy checks, approvals, recovery, and an auditable outbox.
2. **Privacy Plane:** shielded balances, private notes, selective disclosure, clean-provenance proofs, and private DeFi execution.

The result is not merely a wallet, mixer, privacy pool, or treasury dashboard. It is a **private financial operating layer** in which users can:

Describe a goal → simulate the effect → shield or unshield intentionally → check policy and provenance → authorize with quorum → execute atomically → prove or disclose only what is necessary.

Product statement

Veilora lets people and institutions plan, protect, and execute digital-asset actions privately—with no single point of signing authority and no blind movement of funds.

The core user benefit

Veilora should make private finance feel understandable:

- A user sees the before-and-after state.

- The user sees what information leaves the device.
 - The user sees the exact policy and proof requirements.
 - The user explicitly authorizes the action.
 - The user can later prove clean provenance or disclose only selected information.
-

2. Product architecture: three planes

2.1 Control Plane — “What do I want to happen?”

This is the Veilora experience layer and the main differentiation from a conventional privacy pool.

Components:

- Command Center.
- Intent Console.
- Simulation and quote engine.
- Policy and guardrail engine.
- Outbox and approval workflow.
- Recovery and security center.
- Local audit trail.
- Developer SDK for plans, simulations, policies, and approvals.

Responsibilities:

- Convert natural-language goals into structured plans.
- Show balances, fees, routes, timing, and risk before signing.
- Prevent stale, over-limit, or underfunded actions.
- Keep human authorization mandatory in the initial product.
- Make every state and failure explainable.

2.2 Privacy Plane — “Who can see what?”

This is Veilora’s shielded execution layer.

Components:

- Shielded UTXO-style notes.
- Join-split circuits and note commitments.

- Shield / unshield flows.
- Shielded receive addresses.
- View-key and selective-disclosure registry.
- Association-set or clean-provenance proofs.
- Proof-provider / PPOI integration.
- Ragequit or escape path where technically and legally appropriate.

Responsibilities:

- Reduce unnecessary exposure of balances and relationships.
- Permit selective disclosure to an auditor, counterparty, tax tool, or institution.
- Support clean-provenance checks without publishing the user's full book.
- Keep privacy claims specific to the mechanism actually used.

2.3 Execution Plane — “How does it settle?”

This is the chain and protocol adapter layer.

Components:

- Robinhood Chain adapter.
- ERC-4337 account abstraction.
- DEX adapters.
- Bridge and RelayAdapt adapters.
- Morpho lending adapter where supported.
- Uniswap V3/V4-style swap adapter where supported.
- Relayer / broadcaster network.
- Fee and finality quote services.
- Settlement and confirmation monitor.

Responsibilities:

- Execute approved plans on supported networks.
 - Route shielded DeFi actions atomically when possible.
 - Keep chain-specific assumptions out of the core wallet and policy engine.
 - Publish capability declarations for each adapter.
-

3. How the planes work together

Example: shielded stock-token purchase

User request:

“Buy 500 USDG of NVDA exposure and keep it shielded. Do not proceed if the total cost exceeds 0.75%.”

Veilora should:

1. Parse the intent locally.
2. Verify that the token, chain, and shielded route are supported.
3. Retrieve a route and estimate swap, bridge, proving, and execution costs.
4. Simulate the balance diff and expected shielded note output.
5. Check gas reserve, spending limit, fee ceiling, destination policy, and asset eligibility.
6. Show the user what is public, what is shielded, and what third parties are involved.
7. Build an atomic plan: shield USDG → swap → receive shielded stock token.
8. Require explicit human approval.
9. Collect the required threshold signatures.
10. Submit through the execution adapter and track settlement.
11. Save a local receipt with the plan, proof state, route, and final note commitment.
12. Offer a view-key or selective-disclosure action without exposing unrelated positions.

Example: private treasury rebalance

User request:

“Move 20% of idle USDG to Optimism, keep 0.05 ETH reserved for gas, and only use routes under 0.5% total fees.”

Veilora should combine:

- Transparent balance and gas-reserve simulation.
- Bridge route comparison.
- Treasury policy evaluation.
- Outbox review.
- Threshold approval.
- Optional shielded destination or shielded post-settlement position where supported.

The user should not need to understand whether the action uses a transparent ERC-4337 account, a shielded note, a relayer, or a bridge solver unless they open the advanced view.

4. The user experience

4.1 One command center, two visibility modes

Standard mode

For everyday users:

- Send.
- Receive.
- Swap.
- Bridge.
- Shield.
- Unshield.
- Review.
- Recover.

Control mode

For operators and treasury managers:

- Route and proof details.
- Note and commitment metadata.
- Policy versions.
- Disclosure permissions.
- Outbox batch management.
- Relayer and provider selection.
- Transaction graph and settlement status.
- Exportable compliance and audit reports.

4.2 “Enter privacy mode” interaction

The main privacy action should be simple and understandable:

1. **Choose what to protect:** asset, amount, account, or strategy.

2. **Preview the new state:** public balance change, shielded note, fees, and reserve impact.
3. **Review disclosures:** what the chain, co-signer, relayer, and proof provider can see.
4. **Confirm the policy:** clean-provenance and destination checks.
5. **Authorize:** threshold signing and proof generation.
6. **Track:** settlement, note creation, and receipt.

“Shield” should not be a magic button that hides all risk. It should be a clear state transition with a before-and-after explanation.

4.3 Safe unshielding

Unshielding must be designed around failure prevention:

- Default destination is the user’s verified origin account where possible.
- Destination change requires a new review and, for higher-risk flows, a cooling-off period.
- If a proof provider is unavailable, the state should remain explicit: queued, delayed, or blocked.
- No broadcaster should be able to redirect a user’s funds.
- Refund and timeout behavior must be deterministic and documented.

5. Privacy model

Veilora should present privacy as a set of controls, not a binary promise.

5.1 Four privacy layers

Layer	What it protects	What it does not guarantee
Threshold authorization	Signing authority	Does not hide public settlement data
Local planning	Intent text and strategy	Does not hide data required by a chosen network or provider
Shielded notes	Amounts and relationships within the shield	Does not make every endpoint or exit private

Selective disclosure	Unrelated positions and history	Does not make disclosed information private from its recipient
----------------------	---------------------------------	--

5.2 Clean-provenance proofs

A proof-of-innocence or clean-provenance mechanism should be framed as:

- A way to prove inclusion or non-membership against a defined set.
- A way to reduce the need to disclose the entire wallet history.
- A mechanism with provider, set freshness, and coverage limitations.

It must not be marketed as a universal “clean” guarantee.

Required UX fields:

- Provider or association set used.
- Set timestamp and freshness.
- Assets and routes covered.
- Proof validity period.
- What the proof reveals.
- What happens if the provider is unavailable.

5.3 View keys and disclosures

View keys should be scoped and revocable where technically possible:

- Account-level view key.
- Note-level disclosure.
- Time-limited auditor access.
- Tax-period export.
- Counterparty-specific proof.
- Revocation and access history.

A user should be able to see exactly who has access to which information.

6. Shielded asset and DeFi strategy

Phase 1 asset focus

Start with a small, well-tested set:

- USDG.
- USDC or USDT only where the chain adapter supports them correctly.
- Native ETH for gas.
- One or two verified tokenized-stock assets if legal, liquidity, contract, and pricing requirements are satisfied.

Tokenized stock requirements

Tokenized-stock support should not be treated as ordinary ERC-20 support. Each asset needs:

- Issuer and legal wrapper information.
- Transfer restrictions and eligibility rules.
- Dividend or split behavior.
- Non-rebasing or rebase-aware accounting behavior.
- Price oracle and stale-price handling.
- Shielded note denomination behavior.
- Corporate-action treatment.
- Geographic and user eligibility rules where relevant.

Veilora's strongest opportunity is **shielded tokenized-stock activity on Robinhood Chain**, but this should launch only after the asset and legal model are verified.

Shielded DeFi recipes

Use a composable Step → Recipe → Combo structure:

- **Step:** shield, swap, lend, withdraw, bridge, unshield.
- **Recipe:** one reusable, policy-checked workflow.
- **Combo:** atomic multi-step execution with one simulation and one approval.

Each recipe must declare:

- Inputs and outputs.
- Supported assets and chains.
- Contract addresses and versions.
- Fees and slippage behavior.
- Proof requirements.

- Failure and refund behavior.
- Privacy disclosures.
- Test and audit status.

Initial recipe candidates:

1. Shield USDG.
 2. Shield → swap → receive shielded asset.
 3. Shield → lend → monitor yield position.
 4. Unshield → execute → reshield output.
 5. Bridge → receive → shield at destination, where the adapter supports it.
-

7. Threshold custody integration

The original Veilora 2-of-3 architecture remains the authorization backbone.

- **Shard A:** local device or desktop secure storage.
- **Shard B:** policy co-signer or customer-operated co-signer deployment.
- **Shard C:** passkey / recovery device.

Signature and proof order

The system must prevent a proof from being interpreted as approval and prevent a signature from being interpreted as proof.

Recommended order:

1. Build intent.
2. Simulate transparent and shielded effects.
3. Evaluate policy and provenance requirements.
4. Generate or validate proof inputs.
5. Show final plan and disclosures.
6. Obtain explicit user approval.
7. Collect threshold signatures.
8. Submit through a permitted broadcaster or relayer.
9. Verify settlement and note commitments.

Server trust boundary

The co-signer may enforce policy but must not receive more financial information than needed. The product must document:

- What the co-signer sees.
- Whether it sees amounts, recipients, notes, or only commitments.
- What it stores and for how long.
- How a customer can deploy or audit the policy engine.
- What happens if the co-signer is unavailable or malicious.

For institutional adoption, a customer-operated or independently deployable Shard B should be a strategic goal.

8. Broadcaster and prover network

Broadcasters

A permissionless broadcaster network can improve UX by paying gas and submitting eligible actions, but it creates a new economic and failure surface.

Requirements:

- Bonding and slashing rules documented.
- Published fee schedule.
- No authority to redirect destination funds.
- Replay protection.
- Timeout and refund guarantees.
- Route and broadcaster shown before approval.
- Multiple-broadcaster fallback.
- Monitoring for censorship and liveness.

Proving

For mobile and lower-power devices:

- Server-assisted proving may be used with blinded witnesses.
- Local proving remains the highest-privacy option where practical.
- The UI must label where proving occurred.
- Proof providers must not receive raw note contents or unrelated wallet history.

- “Under 10 seconds” is a target, not a promise, until measured across supported devices.

PPOI standby and refund

The proposed 15-minute standby flow is valuable only if the state machine is explicit:

Proof requested → Provider pending → Proof accepted → Action eligible → Timeout / safe refund

The system must never create a “PPOI missing” limbo in which users cannot understand whether funds are safe, delayed, or recoverable.

9. Developer architecture

Veilora should expose a layered SDK rather than forcing developers to understand every privacy primitive.

Core interfaces

Plain Text

```
const plan = await veilora.intent.create({
  goal: "Buy 500 USDG of NVDA exposure and keep it shielded",
  constraints: {
    maxTotalFeeBps: 75,
    preserveGas: "0.05 ETH",
    requireCleanProvenance: true,
  },
});

const simulation = await veilora.simulate(plan);
const policy = await veilora.policies.evaluate(plan, simulation);
const approval = await veilora.approvals.request(plan, simulation, policy);
const result = await veilora.execute(approval);
```

SDK layers

1. **Core:** shards, threshold signing, recovery.
2. **Intent:** natural-language normalization and structured plans.
3. **Simulation:** balance diffs, quotes, fees, slippage, proving, and finality.
4. **Privacy:** notes, shield/unshield, view keys, disclosures, proof state.
5. **Policy:** limits, allowlists, fee ceilings, provenance requirements.

6. **Adapters:** chains, DEXs, bridges, lending protocols, relayers, proof providers.
7. **Observability:** local receipts, redacted diagnostics, and settlement state.

Adapter contract

Every adapter must declare:

- Supported chains and assets.
 - Transparent and shielded capabilities.
 - Fee and quote sources.
 - Proof requirements.
 - Privacy limitations.
 - Finality assumptions.
 - Failure and refund behavior.
 - Test, audit, and version status.
-

10. Launch strategy

Gate 1 — Trustworthy control core

Ship before shielded DeFi:

- 2-of-3 wallet and recovery.
- Local intent planning.
- Simulation and balance diffs.
- Policies and outbox.
- Transparent send, swap, and bridge flows.
- Local audit receipts.

Gate 2 — Shielded payments

- Shield and unshield for one stablecoin.
- Shielded receive flow.
- Selective disclosure.
- Provenance proof integration.
- Safe timeout and refund behavior.

- Mobile proving benchmark.

Gate 3 — Shielded DeFi

- Shielded swap recipe.
- Shielded lending recipe.
- Atomic unshield → execute → reshield where necessary.
- Broadcaster failover.
- Full simulation of privacy and fee effects.

Gate 4 — Tokenized-stock differentiation

- One verified stock-token issuer and asset first.
- Corporate-action and rebase tests.
- Legal and eligibility review.
- Shielded purchase and receipt.
- Proof and selective-disclosure flows.
- Only then expand the registry.

Gate 5 — Multichain privacy

- Add adapters only after the Robinhood Chain privacy and recovery experience is stable.
 - Keep chain-specific privacy assumptions visible.
 - Do not promise identical privacy across chains.
-

11. Claims and language guardrails

Say

- “Private by default where supported.”
- “Prove clean provenance without exposing your full book.”
- “Threshold authorization with explicit human approval.”
- “Shielded execution for supported assets and protocols.”
- “Selective disclosure through scoped view keys.”
- “Private DeFi on Robinhood Chain, subject to adapter support.”

Do not say

- “Untraceable.”
- “Anonymous.”
- “Invisible to everyone.”
- “Guaranteed private.”
- “No one can ever see your transactions.”
- “Risk-free.”
- “Fully decentralized” unless the specific component meets a defined decentralization standard.

Positioning opportunity

Subject to verification, the strongest differentiated claim is:

Veilora is building a private control layer for tokenized assets and DeFi on Robinhood Chain—combining shielded execution with threshold self-custody and explainable intent planning.

This is stronger and more defensible than simply calling Veilora a private wallet.

12. Privacy architecture decisions

Adopt now

- Shield / unshield as a first-class user action.
- Shielded receive addresses.
- View keys and selective disclosure.
- Clean-provenance proof concept.
- Shielded DeFi recipe model.
- Explicit timeout and refund state machine.
- Mobile proving as a product requirement.
- Tokenized-stock support as a strategic differentiator.

Revise before implementation

- “One-tap” privacy: keep the action simple but show the simulation and disclosure summary.

- “Private DeFi” : specify what is shielded, what remains public, and which providers see route data.
- “15-minute PPOI” : make it a measured service-level target with a safe fallback.
- “First privacy system” : use only after market and technical verification.
- Broadcaster subsidies and bonding: define economics, failure handling, and censorship resistance.
- Yield inside the shield: treat as a later recipe with clear smart-contract and protocol risk.

Defer

- Token launch, staking, or fee token mechanics.
- A broad stock-token registry.
- Many-chain shielded support.
- Automated treasury execution.
- Unbounded protocol-composition recipes.
- Any privacy claim that depends on an unaudited or unshipped proof system.

Token design, governance, fee capture, and launch mechanics require a separate product, legal, and security decision. They are outside the scope of this product approach.

13. Success metrics

Safety

- Zero unilateral-spend incidents.
- Zero executions bypassing policy and approval.
- Zero funds trapped by an unresolved proof-provider state.
- Percentage of actions with a viewed simulation.
- Percentage of users completing a recovery drill.

Privacy

- Percentage of eligible local computations completed on-device.
- Number of raw intents sent to external services.
- Percentage of shielded actions with successful note recovery.
- Selective-disclosure completion rate.

- Proof-provider outage recovery time.

Product usefulness

- Time from intent to approved plan.
- Quote-to-settlement discrepancy.
- Shield-to-first-private-action completion rate.
- Successful completion of shield, swap, lend, and unshield recipes.
- Number of avoidable risky actions blocked before signing.

Network and developer health

- Broadcaster liveness and fallback success.
- Proving latency by device class.
- Adapter test coverage.
- SDK integration time.
- Percentage of adapters with current capability declarations.

Do not optimize for shielded TVL or transaction count alone. The product should measure whether users understand and safely control what they do.

14. Final integrated product definition

Veilora is a private financial operating layer for Robinhood Chain and compatible networks. It combines threshold self-custody, intent-based planning, local-first simulation, shielded assets, private DeFi recipes, selective disclosure, clean-provenance proofs, and explicit human authorization into one explainable control loop.

The product's strategic advantage is not any single primitive. It is the integration:

- **The Privacy Plane protects the financial activity.**
- **Veilora controls the decision and authorization.**
- **The execution layer settles the approved plan.**
- **The local receipt explains what happened.**

That integrated approach gives Veilora a more useful product than a standalone privacy pool and a more defensible position than a conventional threshold wallet.